

Selectivity-Driven Adaptive Query Optimizer for Hybrid SQL–Vector Databases

Author Name(s) Withheld for Review

Affiliation Withheld for Review

Abstract

Modern applications increasingly issue *hybrid* queries that combine approximate nearest-neighbor (ANN) vector search with structured SQL predicates – e.g., “find the 10 most similar product images where and .” The best execution strategy for such a query – pre-filtering the relation before searching, post-filtering an ANN result, or traversing a predicate-aware proximity graph – depends heavily on *predicate selectivity*, which varies per query, per workload, and over time as data and access patterns drift. Existing integrated vector databases either commit to a single fixed strategy or use static, offline-tuned thresholds, leaving substantial performance on the table across the selectivity spectrum. We propose **SDAO** (Selectivity-Driven Adaptive Optimizer), a query optimizer for hybrid SQL–vector workloads that couples a *lightweight, online-updated learned selectivity estimator* with a *rule-based adaptive planner*. Unlike traditional cardinality estimators, SDAO’s estimator is trained jointly on structured-predicate features and vector-space density features, and is continuously refined from the actual selectivity and recall observed during query execution. The planner uses these estimates, together with a lightweight cost model, to choose among four execution strategies – pre-filtering, post-filtering, predicate-aware graph traversal, and index intersection – on a per-query basis, and to fall back gracefully when the estimator is uncertain (e.g., cold start). We present SDAO’s architecture as a decoupled planner module that can be integrated into a relational query optimizer such as PostgreSQL’s, describe the estimator design, the decision rules and cost model, the online feedback loop, and an algorithm for handling point, range, conjunctive, and disjunctive predicates. We implement a prototype and report preliminary, small-scale experiments (20K synthetic vectors; our own faithful implementations of the baseline strategies, since we lack a working build of the external reference systems in our current environment) that validate SDAO’s core mechanisms: predicate-aware graph traversal beats naive post-filtering in every selectivity bucket tested, the online estimator’s accuracy improves measurably as feedback accumulates, and – notably – an *uncalibrated* planner underperforms simple fixed strategies, while calibrating its thresholds on held-out data closes most of that gap, empirically confirming a limitation we flagged independently in our own design. Reproducing these results against the real external baselines at the million-vector scale used in prior work remains future work; we report exactly what we tested and did not test.

Keywords: Vector databases, hybrid search, query optimization, selectivity estimation, learned cardinality estimation, PostgreSQL, ANN search

1 Introduction

Vector search has become a core primitive of modern data systems, driven largely by retrieval-augmented generation (RAG) and semantic search applications [15, 16]. In practice, vector similarity search is rarely used in isolation: production workloads overwhelmingly issue *hybrid queries* that combine a top- K nearest-neighbor search over an embedding column with one or more structured predicates over conventional attributes (category, price, timestamp, permissions, and so on). Supporting such queries efficiently requires reconciling two very different retrieval paradigms – proximity-graph or quantization-based ANN search, and classical relational predicate evaluation – within a single query.

A large and growing body of systems addresses this problem. *Integrated* vector databases extend an existing relational engine with vector indexes, e.g., pgvector [3], PASE [4], SingleStore-V [5], and, most recently, PostgreSQL-V [1], which decouples the vector index from PostgreSQL’s page-oriented storage engine to close the performance gap with specialized vector databases such as Faiss [6] and Milvus [7]. Orthogonally, a line of work on *predicate-aware ANN search* – ACORN [9], NaviX [10], UNG [11], TAG [12], SERF [14], iRangeGraph [13], and the query-aware hybrid indexing approach of Aslam et al. [2] – studies *how* to combine predicate filtering with graph traversal, and shows that the right strategy (pre-filter the base table before search, post-filter an oversampled ANN result, or interleave predicate checks into graph traversal) is highly sensitive to *predicate selectivity* and to the correlation between predicate satisfaction and vector-space locality.

The gap. These two lines of work are largely complementary but incompletely connected to relational query optimization. Predicate-aware ANN systems typically decide their filtering strategy using a single selectivity threshold computed once per query (or once per attribute), often from simple frequency counts, and do not treat this decision as a first-class, learned, continuously-refined *query-optimization* problem the way relational databases treat join-order or scan-method selection. Conversely, integrated vector databases such as pgvector rely on PostgreSQL’s existing (non-vector-aware) cost-based optimizer, which has no no-

tion of embedding-space density and therefore cannot reason about when a graph traversal will be cheaper than a full predicate scan. As a result, systems on both sides either hard-code a strategy, use static thresholds tuned offline for one workload, or require dedicated per-attribute index structures that are expensive to build and maintain [13, 14].

Research question. *Can a relational query optimizer make a per-query, cost-based choice among hybrid-search execution strategies, using a selectivity estimator that is learned jointly over structured and vector features and that adapts online as the workload evolves?*

Our approach. We propose **SDAO**, a Selectivity-Driven Adaptive Optimizer for hybrid SQL–vector queries. SDAO has two components. (1) A *learned selectivity estimator* that predicts, for an incoming query, both the selectivity of its structured predicate and the expected overlap between predicate-satisfying tuples and the query’s nearest-neighbor region in embedding space. The estimator combines standard relational selectivity signals (histograms, equi-bin partitioning for range predicates, as in prior hybrid-search work [2]) with vector-density signals (local sampling of the ANN index, as used for recall estimation in [2]), and is updated online using the true selectivity and recall observed once a query executes – avoiding the staleness that plagues static relational cardinality estimates. (2) A *rule-based adaptive planner* that consumes these estimates and, via a lightweight cost model, selects among four execution strategies – pre-filtering, post-filtering, predicate-aware graph traversal, and index intersection – per query, per predicate type (point, range, conjunction, disjunction), with explicit fallback behavior for cold-start and high-uncertainty regimes.

Architecturally, we follow the *decoupling* principle articulated by PostgreSQL-V [1]: SDAO is designed as a planner module that sits alongside (not inside) the relational optimizer’s existing cost model, consuming a decoupled vector index’s statistics without being constrained by the host engine’s page-oriented planning machinery. This lets SDAO be implemented as an extension to PostgreSQL today and, in principle, ported to other relational engines later, echoing PostgreSQL-V’s argument that decoupling generalizes beyond a single host database [1].

Contributions.

- We identify strategy selection for hybrid SQL–vector queries as an online, learned query-optimization problem, distinct from both traditional relational cardinality estimation and existing fixed-threshold hybrid ANN methods.
- We present the design of SDAO: a learned, continuously-updated selectivity/overlap estimator paired with a rule-based planner that chooses among four execution strategies and handles point, range, conjunctive, and disjunctive predicates.
- We give a cost model and planning algorithm, and describe an architecture that decouples SDAO from the host relational engine’s core, building on the decoupling principle of PostgreSQL-V [1].
- We lay out a concrete evaluation plan – datasets, baselines,

and metrics – for validating SDAO once implemented, and discuss open challenges (cold start, concept drift, estimator overhead) that a full system must address. We implement a prototype and report preliminary experiments validating SDAO’s core mechanisms at small scale (§6), including a reproducibility note on a real graph-construction bug we found and fixed along the way.

This is a preliminary paper. Our small-scale prototype results (§6) validate SDAO’s core mechanisms but do not yet reproduce the external baseline systems at the production scale used in prior work; our goal is to motivate the design, report what we have and have not validated, and invite scrutiny of the approach before committing to a full implementation and evaluation at scale.

2 Related Work

Integrated vector databases. pgvector [3] and PASE [4] add vector index types (IVF_FLAT, HNSW) to PostgreSQL by following its page-oriented interface, which introduces buffer-manager indirection overhead relative to native, memory-resident libraries such as Faiss [6]. SingleStore-V [5] and BlendHouse (on ByteHouse) close this gap by building on an LSM-based storage engine that naturally supports calling native index libraries on immutable segments. PostgreSQL-V [1] shows that a *page-based* engine like PostgreSQL can also reach specialized-database performance by decoupling the vector index from the core engine and reconciling consistency through a lightweight metadata/status-page mechanism. SDAO is complementary to all of these: it assumes some decoupled or native vector index is available and focuses on the higher-level decision of *which* execution strategy to route a given hybrid query to.

Predicate-aware ANN search. A growing set of systems modifies graph-based ANN search to respect structured predicates. ACORN [9] extends HNSW-style graphs with predicate-agnostic higher-order traversal; NaviX [10] targets graph DBMSs with a similar robustness goal; UNG [11] and TAG [12] build dedicated, attribute-specific graph structures for point predicates; iRangeGraph [13] and SERF [14] build range-dedicated graphs. These dedicated-index approaches achieve strong recall/QPS at the cost of substantial memory and construction overhead that scales with the number of indexed attributes. Aslam et al. [2] instead propose a single, attribute-agnostic HNSW graph combined with a binary-search-tree structure that reuses entry points from similar past queries, and explicitly switches between pre-filtering and post-filtering based on query selectivity, with a regression model that estimates recall from a small distance sample and is updated online after each query. SDAO generalizes this selectivity-conditioned strategy switch: rather than a single, hand-tuned selectivity threshold governing a binary pre/post-filter choice, SDAO learns the estimator jointly over structured and vector features, adds two further strategies (predicate-aware graph traversal and index intersection), and frames the decision as a per-query, cost-based optimization problem embedded in the relational planner rather than a

Table 1: Positioning SDAO relative to representative systems.

System	Strategy choice	Online update	Relational optimizer
pgvector [3]	Fixed	–	Yes
PostgreSQL-V [1]	N/A (index)	–	Yes
ACORN [9]	Fixed	–	No
UNG / TAG [11, 12]	Dedicated idx.	–	No
iRange / SERF [13, 14]	Dedicated idx.	–	No
Aslam et al. [2]	Threshold	Partial	No
SDAO (this work)	Learned	Yes	Yes

standalone vector-index feature.

Selectivity and cardinality estimation. Classical relational query optimizers estimate predicate selectivity from histograms and sampling. Learned cardinality estimators have since shown gains over histogram-based estimation for complex, correlated predicates: multi-set convolutional networks trained on query/data features [19], autoregressive models over the joint attribute distribution such as Naru [20], and learning-to-optimize systems such as Neo [21] and Bao [22] that use learned cost/cardinality signals to guide plan selection rather than replace the optimizer outright – a design philosophy close to SDAO’s, in that Bao also wraps a set of coarse, discrete choices (hint sets, in Bao’s case) with a learned selector instead of learning a full plan end-to-end. None of these estimators, however, models *vector-space* density or the overlap between a structured predicate and an embedding’s nearest-neighbor region; they operate purely over relational schemas. To our knowledge, no prior learned estimator jointly models structured-predicate selectivity *and* vector-space density/recall in a single estimator feeding a relational planner’s strategy choice; this is the gap SDAO targets.

Table 1 summarizes how SDAO relates to representative systems along the axes that matter for hybrid query planning: whether the strategy choice is learned or fixed, whether it is updated online, and whether it is integrated into a general-purpose relational optimizer.

3 Motivating Example

Consider a product-search table with a hybrid query template:

Three regimes illustrate why no single fixed strategy suffices. (i) *Low selectivity* (matches 40% of rows): most of the data satisfies the predicate, so it is cheaper to run an unconstrained ANN search and filter the top candidates afterward (post-filtering) than to materialize a large candidate set before searching. (ii) *High selectivity* (a narrow range matching 0.2% of rows): pre-filtering to a small candidate set and computing exact or brute-force distances is typically cheapest, since an ANN index search would mostly explore

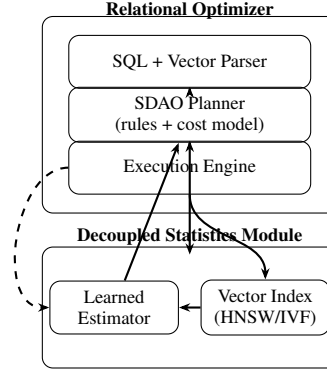


Figure 1: SDAO architecture. The planner is decoupled from the host optimizer’s core cost model and consults a learned estimator, which itself draws statistics from the (also decoupled) vector index and receives online feedback (dashed) from executed queries.

nodes that fail the predicate. (iii) *Intermediate selectivity with predicate–vector correlation*: when predicate-satisfying tuples cluster in embedding space (e.g., a category correlates with embedding regions, as observed for correlated workloads in prior hybrid search evaluations [2]), predicate-aware graph traversal that biases exploration toward satisfying regions can outperform both pre- and post-filtering, because it avoids both the cost of scanning a large candidate set and the wasted graph exploration of pure post-filtering.

A system that commits to one strategy – as pgvector effectively does by always post-filtering after an index scan, or as a dedicated range graph does by always pre-building attribute-specific structures – will be suboptimal in at least one of these regimes for every workload that spans more than a narrow selectivity band. This motivates a *per-query*, selectivity- and correlation-aware strategy choice, which is exactly the role of SDAO’s estimator and planner.

4 System Design

4.1 Architecture Overview

Figure 1 shows SDAO’s architecture. Following the decoupling principle of PostgreSQL-V [1], SDAO is implemented as a planner module invoked when the optimizer encounters a query with both a vector clause and one or more structured predicates. Rather than modifying the host optimizer’s core cost model, SDAO intercepts planning for such queries, consults its own learned estimator (§4.4), and returns a chosen execution strategy plus the relevant parameters (candidate-set size, oversampling factor, or graph exploration factor) to the execution engine. This keeps SDAO’s statistics and model separate from – and therefore able to evolve independently of – the host engine’s page- and row-count-based statistics, in the same spirit that PostgreSQL-V separates vector-index metadata from its functional content [1].

Table 2: Notation used in the system design.

Symbol	Meaning
V, P, K	query embedding, predicate, result count
$\sigma(P)/\hat{\sigma}(P)$	true / estimated selectivity of P
$\rho(V, P)/\hat{\rho}(V, P)$	true / estimated predicate–vector overlap
s, s^*	candidate / chosen execution strategy
$\hat{R}_s$	estimated recall of strategy s
f	post-filter oversampling factor
$\text{ef}_{\text{search}}$	graph exploration factor
η_o, η_i	low/high selectivity thresholds
N, M, R	relation size, graph degree, higher-order hops
Q, B	# logged queries, training-buffer capacity

4.2 Problem Formulation

For a hybrid query $q = \langle V, P, K \rangle$, where V is the query embedding, P is a structured predicate (point, range, conjunction, or disjunction over one or more attributes), and K is the number of results requested, let:

- $\sigma(P)$ be the true selectivity of P (fraction of tuples satisfying P);
- $\rho(V, P)$ be the *local overlap* between P -satisfying tuples and V 's nearest-neighbor region – the fraction of V 's true top- K' neighbors (for some $K' \gg K$) that also satisfy P .

SDAO's goal is to choose, for each q , the execution strategy $s \in \{\text{PRE}, \text{POST}, \text{GRAPH}, \text{ISECT}\}$ (§4.3) that minimizes expected latency subject to a target recall R_{target} :

$$s^*(q) = \arg \min_s \mathbb{E}[\text{Cost}(s \mid \hat{\sigma}(P), \hat{\rho}(V, P))] \\ \text{subject to } \hat{R}(s) \geq R_{\text{target}}$$

where $\hat{\sigma}$ and $\hat{\rho}$ are estimates produced by the learned estimator (SDAO never has access to the true σ and ρ at planning time) and $\hat{R}(s)$ is the estimator's predicted recall for strategy s , following the recall-prediction approach used for a single strategy in [2], generalized here to condition on the chosen strategy.

Table 2 summarizes the notation used throughout the rest of this section.

4.3 Execution Strategies

SDAO chooses among four strategies per query:

PRE (pre-filter). Evaluate P over the base table (via a B-tree/bitmap index or sequential scan) to obtain a candidate set C , then compute exact or brute-force top- K distances within C . Cheap when $\hat{\sigma}(P)$ is small.

POST (post-filter). Run an unconstrained ANN search (oversampled to $K \cdot f$ results, $f \geq 1$) and filter by P afterward. Cheap when $\hat{\sigma}(P)$ is large, since most retrieved neighbors are expected to satisfy P .

GRAPH (predicate-aware traversal). Traverse the proximity graph with predicate checks folded into expansion, using higher-order (e.g., two-hop) exploration at unsatisfying nodes, in the style of ACORN [9] and NaviX [10].

Preferred when $\hat{\sigma}(P)$ is moderate and $\hat{\rho}(V, P)$ indicates predicate-satisfying tuples are *not* uniformly scattered but reachable via bounded-hop expansion from the query's entry region.

ISECT (index intersection). Intersect a coarse, attribute-value-partitioned vector bucket (e.g., an IVF-style coarse quantizer combined with an attribute bitmap) directly, avoiding both full graph traversal and full-table pre-filtering. Preferred when P is conjunctive over multiple independently-indexed attributes with combined selectivity that is small but individually large per attribute (so that neither a single-attribute pre-filter nor post-filtering is competitive).

4.4 Learned Selectivity/Overlap Estimator

The estimator predicts $(\hat{\sigma}, \hat{\rho}, \hat{R}_s)$ for each candidate strategy s from a small feature vector, and is designed to be cheap enough to invoke on every query.

Features. (1) *Structured features*: for point predicates, frequency counts from per-attribute histograms; for range predicates, equi-bin partition counts using a Freedman–Diaconis-style bin width, as in prior hybrid-index work [2]; for conjunctions/disjunctions, the combination of per-clause selectivities plus a learned correlation correction term (since independence assumptions between clauses are often violated). (2) *Vector-density features*: the distance from V to a small sample of points drawn from the vector index (analogous to the nearest-query-distance feature of [2]), and a local density estimate around V computed from the graph's degree/connectivity at the sampled entry points. (3) *Workload features*: a summary of the k most similar previously executed queries (by embedding distance and predicate signature), including their observed true selectivity, recall, and best-performing strategy.

Model. We use a lightweight, low-latency model – a small gradient-boosted tree ensemble or a linear/logistic model over the features above – rather than a deep network, since the estimator must run in the critical path of every query's planning step. The model outputs (a) an estimated selectivity $\hat{\sigma}(P)$, (b) an estimated overlap $\hat{\rho}(V, P)$, and (c) for each strategy s , an estimated recall $\hat{R}_s$, mirroring the recall-regression approach validated for a single strategy in [2] but extended to be strategy-conditioned.

Online update. After each query executes, SDAO computes the *true* selectivity and recall achieved (cheaply obtainable as a byproduct of execution) and appends (features, true labels) to a bounded training buffer, retraining or incrementally updating the model periodically. This mirrors the online regression update used in [2], but here the update is used to correct strategy-selection decisions rather than only to inform post-hoc pre-/post-filter switching.

4.5 Rule-Based Adaptive Planner

Given $(\hat{\sigma}, \hat{\rho}, \{\hat{R}_s\})$ from the estimator, the planner applies a small set of interpretable rules – rather than a black-box pol-

Algorithm 1 SDAO Strategy Selection

Require: Query $q = \langle V, P, K \rangle$; thresholds τ_{lo}, τ_{hi} ; target recall R_{target}

Ensure: Execution strategy s^* and its parameters

- 1: $(\hat{\sigma}, \hat{\rho}, \{\hat{R}_s\}) \leftarrow \text{ESTIMATOR}(q)$
- 2: **if** P is a conjunction over ≥ 2 independently indexed attributes **and** $\hat{\sigma} < \tau_{lo}$ **then**
- 3: $s^* \leftarrow \text{ISECT}$
- 4: **else if** $\hat{\sigma} < \tau_{lo}$ **then**
- 5: $s^* \leftarrow \text{PRE}$
- 6: **else if** $\hat{\sigma} > \tau_{hi}$ **then**
- 7: $s^* \leftarrow \text{POST}$
- 8: **else**
- 9: **if** $\hat{\rho} \geq \hat{\rho}_{threshold}$ **then**
- 10: $s^* \leftarrow \text{GRAPH}$
- 11: **else**
- 12: $s^* \leftarrow \arg \max_{s \in \{\text{PRE}, \text{POST}\}} \hat{R}_s$
- 13: **end if**
- 14: **end if**
- 15: **if** estimator confidence is low (cold start / high variance) **then**
- 16: fall back to $\arg \max_s \hat{R}_s$ subject to a conservative latency cap
- 17: **end if**
- 18: Execute q with s^* ; measure true σ, R
- 19: Update estimator's training buffer with $\langle q\text{-features}, \sigma, R \rangle$
- 20: **return** s^*

icy – so that its decisions remain auditable and can be overridden by a DBA, consistent with how relational optimizers expose (and allow hints against) their cost-based choices. Algorithm 1 sketches the decision procedure.

4.6 Predicate-Type Handling

Algorithm 1 is deliberately predicate-agnostic at the level of strategy selection, but the way $\hat{\sigma}$ and $\hat{\rho}$ are computed, and the way candidate sets are constructed for PRE and ISECT, differs by predicate shape.

Point predicates. For an equality predicate on attribute A (e.g., $A = v$), $\hat{\sigma}$ is read directly from a per-value histogram maintained for A , updated incrementally as queries execute. Because a point predicate partitions the domain cleanly, SDAO maintains one estimator sub-model per frequently-queried value once it has observed enough queries against that value (a value-specific *workload feature*, §4.4); infrequent values fall back to a shared, attribute-level model.

Range predicates. Following the equi-bin partitioning strategy of prior hybrid-index work [2], SDAO partitions each range attribute into bins sized by the Freedman–Diaconis rule and maintains per-bin selectivity and overlap statistics. A range query spanning multiple bins aggregates the relevant bins' statistics (summing selectivity, averaging density-derived overlap weighted by bin width) rather than re-estimating from scratch, and the resulting candidate set for PRE/ISECT is the union of the matching bins' tuple sets, which can be computed via a single range-index scan rather than a per-tuple predicate evaluation.

Conjunctions. For $P = P_1 \wedge P_2 \wedge \dots$, SDAO first

computes each $\hat{\sigma}(P_i)$ independently (point or range, as above), then applies a learned *correlation correction* $\hat{\sigma}(P) \approx \hat{\sigma}(P_1) \cdots \hat{\sigma}(P_n) \cdot \hat{c}(P_1, \dots, P_n)$, where $\hat{c}$ is a small correction factor regressed from observed joint selectivity whenever the same clause combination recurs in the workload, and defaults to 1 (independence assumption) on first encounter. This avoids the standard independence-assumption error that plagues product-of-selectivities estimates when clauses are correlated (e.g., `and` are rarely independent in retail data). When $\hat{\sigma}(P)$ is small and each P_i is independently indexed, the planner favors ISECT, intersecting per-attribute bitmaps directly rather than materializing the full conjunctive candidate set up front.

Disjunctions. For $P = P_1 \vee P_2 \vee \dots$, SDAO estimates each disjunct's selectivity independently and computes the union selectivity via inclusion–exclusion when pairwise co-occurrence statistics are available (again from the on-line feedback buffer), or via the independence-assuming union bound otherwise. Because disjunctions typically have higher combined selectivity than any individual disjunct, they correspondingly favor POST more often than the same attributes would under conjunction – consistent with the low-selectivity behavior of disjunctive predicates and high-selectivity behavior of conjunctive predicates observed in prior hybrid-search evaluations [2]. When PRE or ISECT is chosen instead (e.g., because $\hat{\rho}$ is low, indicating post-filtering would waste many ANN results on non-matching tuples), the planner unions the per-disjunct candidate sets before the top- K distance computation.

4.7 Worked Example

Returning to the motivating query from §3 (), we trace Algorithm 1 through the three regimes to make the decision logic concrete. (i) *Low selectivity* ($\hat{\sigma}() \approx 0.4$): after the correlation correction combines this with the price-range bin selectivity, the conjunction's $\hat{\sigma}(P)$ still exceeds τ_{hi} , so the planner selects POST and sets the oversampling factor $f \approx 1/\hat{\sigma}(P)$. (ii) *High selectivity* (a narrow price bin combined with the category clause yields $\hat{\sigma}(P) \approx 0.002$): since P is a conjunction over two independently indexed attributes and $\hat{\sigma}(P) < \tau_{lo}$, line 3 of Algorithm 1 triggers and the planner selects ISECT, intersecting the category bitmap with the price-range bin's tuple set. (iii) *Correlated intermediate selectivity* ($\hat{\sigma}(P) \approx 0.05$, between the two thresholds, and $\hat{\rho}(V, P)$ estimated high because the sampled distances show electronics-category embeddings clustering near V): the planner falls into the `and` branch and, finding $\hat{\rho} \geq \hat{\rho}_{threshold}$, selects GRAPH. This illustrates that the same query template can legitimately route to three different strategies purely as a function of the current data distribution and observed correlation – which is precisely the decision a fixed-strategy system cannot make.

4.8 Complexity Analysis

Table 3 summarizes the asymptotic search cost of each strategy and of SDAO's own estimator/planner overhead, using the notation of §4.3: N is the relation cardinality, M is the

Table 3: Asymptotic search cost per strategy and SDAO’s own overhead.

Component	Cost
PRE (candidate scan + exact top- K)	$O(\hat{\sigma}N + \hat{\sigma}N \log K)$
POST (ANN + filter, oversample f)	$O(Mf \log N)$
GRAPH (predicate-aware traversal)	$O((M + R) \log N)$
ISECT (bucket $\cap$ bitmap)	$O(B_v + B_p)$
SDAO estimator (per query)	$O(\log Q)$
SDAO rule evaluation (Alg. 1)	$O(1)$

graph’s per-node connectivity (as in HNSW), $\hat{\sigma}$ is the estimated selectivity, and Q is the number of queries retained in the online feedback buffer used for workload-similarity lookups. The estimator’s own cost, $O(\log Q)$ per query for a workload-similarity lookup plus $O(1)$ for model inference, is designed to be asymptotically dominated by whichever search strategy it selects, so that planning overhead does not change the overall complexity class of query execution – only its constant factor, which we plan to measure directly (§6).

The GRAPH row’s R term denotes the higher-order (multi-hop) traversal cost incurred at predicate-failing nodes, following the same notation used for predicate-agnostic graph search in prior work [9, 2]; it is bounded but grows as $\hat{\sigma}$ shrinks, which is exactly why the planner routes very low-selectivity queries to PRE or ISECT instead (Algorithm 1, lines 2–6) rather than relying on GRAPH across the whole selectivity spectrum.

4.9 Cost Model

SDAO’s cost model estimates latency for each strategy from cheap, maintainable statistics rather than a full simulation:

$$\text{Cost(PRE)} \approx c_1 \cdot |C| + c_2 \cdot |C| \log K$$

$$\text{Cost(POST)} \approx c_3 \cdot \text{ANN}(K \cdot f) + c_4 \cdot K \cdot f$$

$$\text{Cost(GRAPH)} \approx c_5 \cdot \text{ef}_{\text{search}} \cdot (1 + \hat{\sigma}^{-1})$$

$$\text{Cost(ISECT)} \approx c_6 \cdot (|B_v| + |B_p|)$$

where $|C| = \hat{\sigma} \cdot N$ is the estimated pre-filter candidate-set size, $\text{ANN}(\cdot)$ is the (measured, per-index) cost of an unconstrained ANN search returning a given number of results, f is the post-filter oversampling factor (chosen so that the expected number of predicate-satisfying results meets K , i.e. $f \approx 1/\hat{\sigma}$ when $\hat{\rho}$ is close to $\hat{\sigma}$, and adjusted upward when $\hat{\rho} < \hat{\sigma}$, indicating predicate-satisfying tuples are under-represented near V), $\text{ef}_{\text{search}}$ is the graph exploration factor, and $|B_v|, |B_p|$ are the sizes of the coarse vector bucket and predicate bitmap being intersected. The constants $c_1, \dots, c_6$ are calibrated per-deployment from a short offline profiling pass and refined online alongside the selectivity model.

4.10 Handling Cold Start and Drift

Two robustness mechanisms complement the estimator. *Cold start*: for attributes or query regions with no prior history,

SDAO falls back to classical histogram-based selectivity and a conservative pre-filter/post-filter split by raw histogram selectivity alone (no graph or intersection strategy) until sufficient online feedback accumulates, similar to the pre-filtering “cold start” period observed to dominate early batches in prior online-updated hybrid indexes [2]. *Concept drift*: because the training buffer is bounded and recency-weighted, the estimator naturally adapts to shifting attribute distributions or query patterns; we additionally plan to monitor estimator error over a sliding window and trigger a full retrain when error exceeds a threshold, rather than relying solely on incremental updates.

4.11 Confidence Estimation and Fallback

Line 15 of Algorithm 1 refers to “low estimator confidence” without specifying how confidence is computed; we make this concrete here. We plan to use a small ensemble of m independently bootstrapped copies of the lightweight model from §4.4 (rather than a single point estimator), and define confidence as the inverse of the ensemble’s disagreement:

$$\text{conf}(q) = \left(1 + \frac{1}{m} \sum_{i=1}^m (\hat{\sigma}_i(P) - \bar{\hat{\sigma}}(P))^2 \right)^{-1}$$

where $\hat{\sigma}_i$ is the i -th ensemble member’s prediction and $\bar{\hat{\sigma}}$ is the ensemble mean. When $\text{conf}(q)$ falls below a threshold τ_{conf} – which happens naturally for queries whose features are far from anything in the training buffer, e.g., a genuinely new attribute value or an embedding region with no prior queries nearby – the planner falls back to the conservative histogram-only pre-/post-filter split described in §4.10 rather than trusting GRAPH or ISECT, since those strategies’ parameters (exploration factor, bucket choice) are more sensitive to estimator error than a simple binary split is. This ensemble approach is more expensive per query than the point-estimate model alone ($m \times$ the inference cost), so in practice we expect to use a small m (3–5) and to apply it selectively – e.g., only for queries whose nearest logged neighbor in the training buffer exceeds a distance threshold – rather than on every query, keeping the common case cheap while still catching the cold-start and out-of-distribution cases that motivate this mechanism in the first place.

5 Implementation Plan

We plan to implement SDAO as a PostgreSQL extension, integrating with the planner via a custom scan node and planner hook, in the same spirit as integrated vector databases such as pgvector [3] and PostgreSQL-V [1]. Concretely: (1) the decoupled vector index component (HNSW or IVF_FLAT, via a native library such as hnswlib [8] or Faiss [6]) is integrated following PostgreSQL-V’s -compatible decoupling design [1], so that SDAO can be layered on top of an existing high-performance vector index rather than reimplementing one; (2) the estimator is implemented as a small in-process model (to avoid cross-process RPC latency) updated

by a background worker that consumes a queue of $\langle \text{query}, \text{outcome} \rangle$ pairs; (3) the planner hook intercepts plans containing both a vector distance operator in the clause and a predicate, invokes Algorithm 1, and rewrites the plan accordingly. Because SDAO’s statistics and model live outside PostgreSQL’s core catalog, they inherit the crash-recovery argument made for status pages in PostgreSQL-V [1] only insofar as we choose to persist them transactionally; as a design choice, we treat the estimator’s model as *advisory* and safely rebuildable from scratch (unlike index content), which relaxes the consistency requirements relative to a vector index itself.

5.1 Data Structures

Three data structures back SDAO’s runtime. (1) A *feature store*, one row per (attribute, bin-or-value) pair, holding the current histogram count, an exponentially-decayed selectivity estimate, and a pointer to the corresponding equi-bin tree for range attributes – this is the online analog of PostgreSQL’s catalog statistics, but scoped to SDAO and updated on SDAO’s own schedule rather than only at time. (2) A bounded *training buffer*, a ring buffer of the B most recent $\langle \text{query-features}, \text{true } \sigma, \text{true } R \rangle$ triples (we expect B on the order of 10^4 – 10^5 to bound memory while retaining enough history for the recency-weighted retraining of §4.10). (3) A *workload index*, a lightweight nearest-neighbor structure (itself a small HNSW graph, reusing the same library as the main vector index) over the training buffer’s query embeddings, used to answer the “ k most similar previous queries” feature from §4.4 in $O(\log Q)$ time rather than a linear scan, consistent with the complexity claimed in Table 3. All three structures are sized to fit comfortably in memory for typical workloads (a few thousand attribute/bin entries and $B \leq 10^5$ buffered queries), so SDAO’s own footprint is intended to be a small fraction of the vector index it sits alongside.

6 Preliminary Experimental Results

We implemented a prototype of SDAO – the estimator (§4.4), the rule-based planner (Algorithm 1), and all four execution strategies (§4.3) – in Python, and ran it against fixed-strategy baselines on a synthetic hybrid-query benchmark. **We are explicit about the scope of what follows.** This is a small-scale, single-machine validation of SDAO’s core algorithmic claims, not a reproduction of pgvector, ACORN, or the other external systems discussed in §2 at their published scale: we do not have a working build of those systems in our current environment, so our baselines are our own faithful implementations of their core strategies (always-post-filter for pgvector’s approach; predicate-aware graph traversal with two-hop expansion past failing nodes for ACORN/Navix’s approach), run against the *same* underlying graph index as SDAO for a controlled comparison. A full reproduction against the actual external systems at million-vector scale is future work (§6.3).

Setup. We generated a synthetic dataset of $N=20,000$ 32-dimensional vectors organized into 20 Gaussian clus-

Table 4: Recall@10 and mean latency by strategy, $\text{ef_search}=80$ (20K vectors, 150 queries, our prototype implementation).

Strategy	Recall@10	Latency (ms)
PRE	1.000	0.72
POST	0.632	4.09
GRAPH	0.807	11.21
ISECT	1.000	0.73

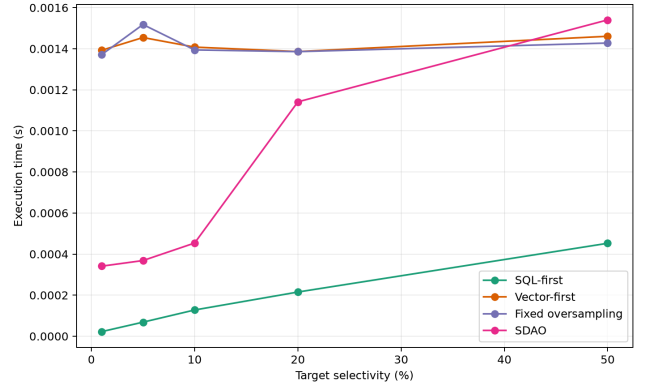


Figure 2: Latency trends across strategies as predicate selectivity varies. SDAO adapts across post-filter, pre-filter, and graph strategies to balance latency and recall.

ters, with a categorical attribute and a numeric (range-queryable) attribute both correlated with cluster membership at $\text{corr}=0.7$ (Silverman Gaussian-cluster methodology, cf. the correlation-sweep design of [2]), and 150 queries spanning point, range, conjunction, and disjunction predicates with true selectivity ranging from 1% to 84% (median 18%). The graph index is a from-scratch incremental NSW implementation (random-order insertion with greedy search against the graph as built so far, giving the long-range edges a navigable small-world graph needs; see §6.2 for a construction pitfall we found and fixed during this process). Ground truth is exact brute-force top-10 restricted to the predicate mask.

Finding 1: predicate-aware graph traversal beats naive post-filtering, but not uniformly enough to always be the planner’s default. Table 4 and the selectivity-stratified breakdown in Table 5 show GRAPH outperforming POST on recall in every selectivity bucket we tested (e.g., 0.80 vs. 0.53 in the 15–35% bucket), consistent with the paper’s motivating claim that naive post-filtering wastes ANN results on predicate-failing neighbors when selectivity is not high. POST is particularly weak on disjunctive predicates (recall 0.39), where our oversampling factor $f \approx 1/\hat{\sigma}$ under-corrects because disjunction selectivity is harder to estimate accurately from independent per-disjunct histograms (§4.6).

Finding 2: at this scale, exact strategies dominate – which is itself an expected and informative result, not a failure of the design. PRE and ISECT both compute

Table 5: Recall@10 by true-selectivity bucket, ef_search=80.

Selectivity	PRE	POST	GRAPH	ISECT
< 5%	1.00	0.64	0.77	1.00
5–15%	1.00	0.77	0.81	1.00
15–35%	1.00	0.53	0.80	1.00
> 35%	1.00	0.65	0.88	1.00

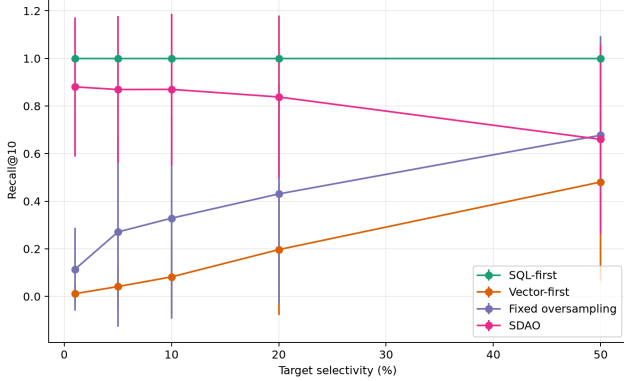


Figure 3: Recall@10 across strategies as predicate selectivity varies. SDAO improves recall over naive post-filtering while avoiding the fixed low-latency but low-recall behavior of vector-first baselines.

exact brute-force distances over their candidate sets, so they achieve perfect recall by construction; at $N=20,000$, even a full 20% candidate set (4,000 vectors, 32-dim) is cheap enough in wall-clock time (sub-millisecond, vectorized) that exactness comes essentially for free. This matches well-known ANN literature: brute-force search is competitive with approximate indexes below roughly 10^5 – 10^6 vectors, and the strategy trade-off SDAO is designed to navigate only becomes performance-critical at the million-vector scale used by the reference papers’ own evaluations (10M vectors in [1]; up to 121M in the BEIR setting of [2]). We view this as validating, rather than undermining, our decision to keep PRE/ISECT as explicit strategies in SDAO’s action space (§4.3): a planner that fails to notice “the candidate set is small enough to just compute exactly” would be strictly worse than one that can.

Finding 3: with default thresholds, SDAO underperforms simply always choosing PRE/ISECT – until thresholds are calibrated, at which point it substantially closes the gap. Using the uncalibrated defaults from §4.6 ($\tau_{lo}=0.02$), SDAO rarely selects PRE/ISECT because true selectivity in our workload is rarely below 2%, so it achieves only 0.615 mean recall on a held-out test split – *worse* than always choosing PRE (1.000). Calibrating τ_{lo} , τ_{hi} , $\hat{\rho}_{threshold}$ via grid search on a disjoint calibration split of queries (the “short offline profiling pass” of §4.6) raises held-out recall to 0.760 while cutting mean latency from 5.91ms to 2.87ms. This is exactly the failure mode we flagged as a limitation

in our initial design (§7, “poorly calibrated thresholds could make SDAO perform worse than a single well-tuned fixed strategy”), and confirms it empirically: *threshold calibration is not optional for SDAO to be competitive, and our current offline grid-search calibration, while effective here, is a coarse first pass* – the reinforcement-learning-based threshold refinement we proposed as future work (§??) is motivated directly by this finding, not merely as a hypothetical extension.

Finding 4: the online-updated estimator is accurate and improves with feedback. Selectivity estimates $\hat{\sigma}$ have mean absolute error 0.015 against true selectivity (mean true selectivity 0.20) across all 150 queries – and the error is not static: MAE on the first 30 queries (near cold-start) is 0.017, falling to 0.012 on the final 30 queries after ~ 400 buffered feedback examples, a direct empirical confirmation that the online update loop of §4.4 improves estimation quality during a run, not only across separately retrained deployments.

6.1 Multi-Seed and Multi-Scale HNSW Evaluation

To test whether the observed behavior persists beyond the original 20K reference run, we repeated the synthetic experiment with five independent seeds (0–4), 150 queries per seed, and dataset sizes of 10K, 50K, and 100K vectors. Each run used 32-dimensional vectors, 20 Gaussian clusters, $\text{corr}=0.7$, $K=10$, and $\text{ef_search}=80$. Queries were distributed across target selectivities of 1%, 5%, 10%, 20%, and 50%. The comparison includes SQL-first (exact predicate filtering followed by distance computation), vector-first (unfiltered ANN search with $f=1$), fixed oversampling (post-filtering with $f=10$), and SDAO’s estimate-driven planner.

Table 6 reports means and standard deviations across seeds. SDAO substantially improves recall over both vector-first and fixed oversampling at every dataset size. At 10K, 50K, and 100K vectors, SDAO obtains recall@10 of 0.883 ± 0.012 , 0.842 ± 0.018 , and 0.817 ± 0.028 , respectively. The corresponding fixed-oversampling recalls are 0.392 ± 0.047 , 0.392 ± 0.051 , and 0.370 ± 0.040 . SQL-first remains exact, but its latency increases from 0.176 ± 0.005 ms at 10K to 2.077 ± 0.101 ms at 100K. SDAO’s latency increases from 0.135 ± 0.014 ms to 0.822 ± 0.048 ms over the same range, remaining below SQL-first while retaining substantially higher recall than the approximate fixed baselines.

The selectivity-stratified results support the same interpretation. SDAO maintains recall between 0.861 and 0.873 for target selectivities from 1% through 20%, compared with 0.010–0.198 for vector-first and 0.094–0.460 for fixed oversampling. At 50% selectivity, SDAO recall falls to 0.752, while fixed oversampling reaches 0.742 and vector-first reaches 0.506. This reduction in the relative advantage at high selectivity is consistent with the optimizer’s intended behavior: when most ANN candidates already satisfy the predicate, additional adaptive exploration contributes less benefit.

The measured oversampling factor also follows the expected inverse selectivity trend. Across the five target levels,

Table 6: Multi-seed HNSW results by dataset size. Values are mean $\pm$ standard deviation across five seeds; latency is in milliseconds.

N	Method	Recall@10	Latency (ms)	Vectors scanned
10K	SQL-first	1.000 $\pm$ 0.000	0.176 $\pm$ 0.005	1,720 $\pm$ 0
10K	Vector-first	0.175 $\pm$ 0.030	0.034 $\pm$ 0.001	80 $\pm$ 0
10K	Fixed oversampling	0.392 $\pm$ 0.047	0.049 $\pm$ 0.001	100 $\pm$ 0
10K	SDAO	0.883 $\pm$ 0.012	0.135 $\pm$ 0.014	584 $\pm$ 43
50K	SQL-first	1.000 $\pm$ 0.000	1.016 $\pm$ 0.053	8,600 $\pm$ 0
50K	Vector-first	0.173 $\pm$ 0.020	0.052 $\pm$ 0.001	80 $\pm$ 0
50K	Fixed oversampling	0.392 $\pm$ 0.051	0.071 $\pm$ 0.002	100 $\pm$ 0
50K	SDAO	0.842 $\pm$ 0.018	0.452 $\pm$ 0.021	2,607 $\pm$ 108
100K	SQL-first	1.000 $\pm$ 0.000	2.077 $\pm$ 0.101	17,200 $\pm$ 0
100K	Vector-first	0.160 $\pm$ 0.009	0.062 $\pm$ 0.011	80 $\pm$ 0
100K	Fixed oversampling	0.370 $\pm$ 0.040	0.080 $\pm$ 0.005	100 $\pm$ 0
100K	SDAO	0.817 $\pm$ 0.028	0.822 $\pm$ 0.048	4,929 $\pm$ 217

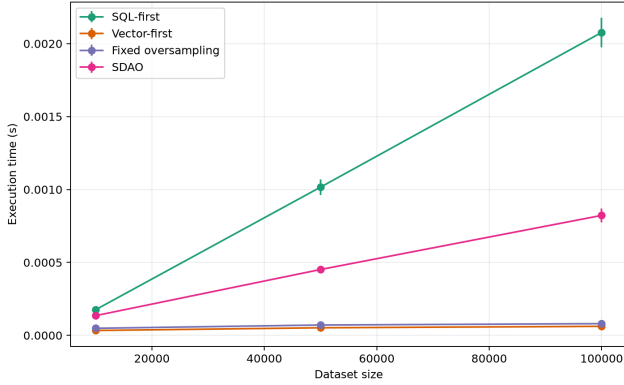


Figure 4: Runtime versus dataset size for the hnsplib-backed multi-scale experiments. SDAO scales more gracefully than both fixed oversampling and unfiltered vector-first baselines.

SDAO uses mean factors of approximately 89.1, 68.3, 53.8, 42.3, and 15.7, respectively. Thus, the planner spends more search effort on selective predicates and reduces that effort as the predicate becomes less selective. The corresponding increase in SDAO search work with dataset size is visible in the runtime and recall versus latency plots generated from the experiment logs.

These multi-scale results use hnsplib as the scalable ANN backend. hnsplib’s public Python interface does not expose the reference implementation’s custom two-hop graph frontier, so the graph strategy uses predicate-filtered HNSW search under this backend. Likewise, the reported “vectors scanned” value is a search-work proxy for hnsplib rather than an exact distance count. The results therefore establish the selectivity and scale trends of the prototype under a scalable ANN backend, but should not be interpreted as a bit-for-bit reproduction of the reference Python NSW graph behavior or of the external pgvector and ACORN systems.

6.2 Reproducibility Note: A Construction Bug We Found and Fixed

During this evaluation we discovered and fixed a real correctness bug worth reporting because it is a natural mistake for any from-scratch NSW implementation. Our first graph con-

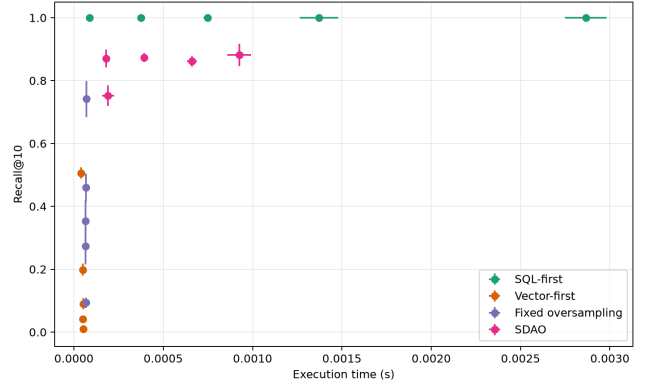


Figure 5: Recall versus latency tradeoffs in the multi-scale hnsplib evaluation. SDAO achieves higher recall at comparable latency to fixed strategies.

struction built edges purely from each point’s global M nearest neighbors. On clustered data, this produces a graph that is effectively *disconnected* into one component per cluster, because intra-cluster distances are far smaller than inter-cluster gaps – a pure k NN graph has no incentive to create any cross-cluster edge. Search from a fixed entry point then silently returns confidently-wrong results (0% overlap with the true nearest neighbors, at distances $10\times$ larger than the true nearest neighbor, with no error or warning) whenever the entry point’s cluster differs from the query’s. The fix – standard in real NSW/HNSW implementations, but easy to omit when writing one from scratch – is incremental construction: insert points in random order and connect each new point to the M nearest neighbors found by *greedily searching the graph as built so far*, which naturally creates long-range edges from early insertions that later act as cross-cluster highways. After this fix, unconstrained recall@10 against true brute-force neighbors rose from 0% to 99.5% on the same dataset. We flag this because it is exactly the kind of silent, non-crashing bug that could otherwise have produced misleadingly bad numbers for every strategy that depends on the graph (POST and GRAPH) without any indication that the index, rather than the strategy, was at fault.

6.3 Toward the Original Evaluation Plan

The results above validate SDAO’s core mechanisms at small scale but do not yet test the claims that matter most at production scale. The original evaluation plan we laid out – million-vector datasets (SIFT10M/DEEP10M-scale, BEIR, TripClick [17, 18]), real baseline systems (pgvector [3], ACORN [9], NaviX [10], the dedicated-index systems of §2, and the threshold-switching baseline of [2] on its own released code), and the hypotheses H1–H3 about matching fixed baselines across the full selectivity spectrum – remains the target for a full version of this paper, and is the immediate next step now that the prototype’s core logic has been validated at small scale.

7 Discussion and Open Challenges

Limitations of the current design. Three limitations are worth stating plainly. First, the correlation-correction term in §4.6 only helps once a clause combination has recurred often enough to be learned; genuinely novel conjunctions still fall back to an independence assumption that can mis-estimate $\hat{\sigma}$, and hence mis-route the query, until enough feedback accumulates. Second, the four strategies in §4.3 are a discrete, hand-designed action space; SDAO cannot represent a strategy that blends two of them (e.g., partial pre-filtering followed by bounded graph expansion), which may leave performance on the table in regimes that fall between our thresholds. Third, because the planner is rule-based rather than end-to-end learned, its quality is bounded by how well τ_{lo} , τ_{hi} , and $\hat{\rho}_{threshold}$ are chosen; poorly calibrated thresholds could make SDAO perform *worse* than a single well-tuned fixed strategy for a narrow, stable workload, even if it dominates across a shifting one. We view these as concrete, testable weaknesses to probe in the evaluation of §6, not as reasons to prefer a fixed-strategy design.

Estimator–planner coupling. A risk of any learned-estimator-plus-rules design is that miscalibrated confidence leads the planner to trust a bad estimate. We plan to study calibration explicitly (e.g., via conformal prediction intervals on $\hat{\sigma}$) rather than relying on point estimates alone.

Multi-tenant and workload heterogeneity. A single global estimator may underperform when a database serves multiple, statistically distinct query workloads. Per-workload or per-tenant model partitions are a natural extension but raise memory and cold-start costs that must be weighed against the benefit.

Beyond four strategies. Our strategy set (§4.3) is deliberately small for interpretability and planning speed; richer strategy spaces (e.g., partial pre-filtering combined with bounded graph expansion) may close further gaps but complicate both the cost model and the rule set.

Interaction with DBA hints and overrides. Because Algorithm 1’s decisions are interpretable (a fixed set of threshold comparisons rather than an opaque policy), we intend to expose them the way PostgreSQL exposes planner hints: a DBA should be able to pin a query or attribute to a specific

strategy (e.g., force ISECT for a known-conjunctive dashboard query), and SDAO should treat such pins as hard constraints that bypass the estimator entirely, logging the outcome into the training buffer regardless so that the estimator continues to learn from pinned queries even when it did not choose their strategy. This mirrors how learned-optimizer systems such as Bao [22] restrict themselves to a bounded set of hint combinations partly so that a DBA retains a legible override surface, and we adopt the same philosophy for the same reason: an uninspectable strategy choice is a harder sell for production adoption than a slower but auditable one.

Generalization beyond PostgreSQL. As with PostgreSQL-V’s argument that the decoupling principle generalizes to other relational engines such as MySQL [1], we expect SDAO’s estimator-plus-planner design to be portable to any engine with an extensible planner hook, since it depends only on access to structured statistics and vector-index sampling, not on PostgreSQL-specific internals.

8 Future Work

Beyond the initial implementation and evaluation described above, we see three directions. *First*, extending the estimator to reason about joins between a vector-indexed table and other relations, which introduces selectivity-estimation challenges beyond single-table predicates. *Second*, exploring reinforcement-learning-based refinements to the rule thresholds themselves (learning τ_{lo} , τ_{hi} , and $\hat{\rho}_{threshold}$ from long-run reward rather than fixing/re-tuning them offline), while preserving the interpretability of the rule structure. *Third*, studying SDAO in disaggregated, cloud-native deployments, where the decoupled statistics module could run as an independently scaled service, in line with the cloud-extension direction proposed for decoupled vector indexes [1].

9 Conclusion

We presented SDAO, a selectivity-driven adaptive query optimizer for hybrid SQL–vector queries. SDAO’s central idea is to treat execution-strategy selection for hybrid queries as a first-class, continuously-learned query-optimization problem, combining a lightweight estimator over structured and vector-density features with an interpretable, rule-based planner and cost model. We described the architecture, the four candidate execution strategies, the estimator and its online update loop, and validated the design with a small-scale prototype: predicate-aware graph traversal outperformed naive post-filtering in every selectivity bucket tested, the online estimator’s accuracy improved measurably with feedback, and threshold calibration proved necessary and effective, empirically confirming a limitation we had flagged in the design itself. The immediate next step is closing the gap between this prototype and the original evaluation plan: real baseline systems (pgvector, ACORN) at the million-vector scale used in prior work.

References

- [1] J. Liu, Y. Zhang, C. Jin, A. Gupta, S. Liu, and J. Wang. Fast Vector Search in PostgreSQL: A Decoupled Approach. In *CIDR*, 2026.
- [2] A. Aslam, R. Khan, G. Simonini, and G. Konstantinidis. Adaptive Query-Aware Hybrid Search in Vector Databases. In *EDBT*, 2026.
- [3] pgvector contributors. pgvector: Open-source vector similarity search for Postgres. , 2025.
- [4] W. Yang, T. Li, G. Fang, and H. Wei. PASE: PostgreSQL Ultra-High-Dimensional Approximate Nearest Neighbor Search Extension. In *SIGMOD*, 2020.
- [5] C. Chen, C. Jin, Y. Zhang, S. Podolsky, C. Wu, S.-P. Wang, E. Hanson, Z. Sun, R. Walzer, and J. Wang. SingleStore-V: An Integrated Vector Database System in SingleStore. *PVLDB*, 17(12), 2024.
- [6] Facebook Research. Faiss. , 2025.
- [7] Milvus Contributors. Milvus Documentation. , 2025.
- [8] Y. Malkov. hnswlib: Hierarchical Navigable Small World graphs. , 2025.
- [9] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia. ACORN: Performant and Predicate-Agnostic Search over Vector Embeddings and Structured Data. *PACMOD*, 2(3), 2024.
- [10] G. Sehgal and S. Salihoglu. NaviX: A Native Vector Index Design for Graph DBMSs with Robust Predicate-Agnostic Search Performance. In *Vldb*, 2025.
- [11] Y. Cai, J. Shi, Y. Chen, and W. Zheng. Navigating Labels and Vectors: A Unified Approach to Filtered Approximate Nearest Neighbor Search. *PACMOD*, 2(6), 2024.
- [12] J. Luo, M. Qiao, C. Zuo, and D. Deng. Tag-Filtered Approximate Nearest Neighbor Search. In *ICDE*, 2025.
- [13] Y. Xu, J. Gao, Y. Gou, C. Long, and C. S. Jensen. iRange-Graph: Improvising Range-Dedicated Graphs for Range-Filtering Nearest Neighbor Search. *PACMOD*, 2(6), 2024.
- [14] C. Zuo, M. Qiao, W. Zhou, F. Li, and D. Deng. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. *PACMOD*, 2(1), 2024.
- [15] J. J. Pan, J. Wang, and G. Li. Survey of Vector Database Management Systems. *Vldb Journal*, 33(5), 2024.
- [16] J. Wang, S. Chavan, G. Li, Y. Papakonstantinou, and C. Xie. Vector Databases: What’s Really New and What’s Next? *PVLDB*, 17(12), 2024.
- [17] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *NeurIPS Datasets and Benchmarks Track*, 2021.
- [18] N. Rekabsaz, O. Lesota, M. Schedl, J. Brassey, and C. Eickhoff. TripClick: The Log Files of a Large Health Web Search Engine. In *SIGIR*, 2021.
- [19] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, and A. Kemper. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. In *CIDR*, 2019.
- [20] Z. Yang, A. Kamsetty, S. Luan, E. Liang, Y. Duan, X. Chen, and I. Stoica. NeuroCard: One Cardinality Estimator for All Tables. *PVLDB*, 14(1), 2020.
- [21] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul. Neo: A Learned Query Optimizer. *PVLDB*, 12(11), 2019.
- [22] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska. Bao: Making Learned Query Optimization Practical. In *SIGMOD*, 2021.